CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
An Autonomous Institute | Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in
COMMITTED TO RESEARCH, INNOVATION AND EDUCATION
48 years

Sign Language Recognition and Text Generation Using CNN-LSTM with Attention

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)

(Affiliated to Osmania University)

CASE STUDY REPORT

Submitted By:

160124748095 – B. Srijay Ram Reddy
160124748098 – B. Shashi Nayak

CASE STUDY GUIDE

Panigrahi Srikanth
Assistant Professor
Department of CSE AI&ML
CBIT, Hyderabad

Department of CSE Artificial Intelligence and Machine Learning
CBIT, Hyderabad

Certificate

This is to certify that the project report entitled **Sign Language Recognition and Text Generation Using CNN-LSTM with Attention** has been carried out by the following students during the academic year 2026–2027 under my guidance and supervision.

160124748095 – B. Srijay Ram Reddy
160124748098 – B. Shashi Nayak

Project Guide:

Panigrahi Srikanth
Assistant Professor
Department of CSE AI&ML
CBIT, Hyderabad

Acknowledgement

We express our sincere gratitude to everyone who supported us throughout the completion of this case study.

We thank our guide, **Panigrahi Srikanth** sir, for his continuous guidance, valuable suggestions, encouragement, and technical insights throughout the development of this work.

We also thank the Head of the Department, faculty members, and the Department of CSE Artificial Intelligence and Machine Learning, Chaitanya Bharathi Institute of Technology, for providing the necessary academic environment and resources required to carry out this case study.

Finally, we express our sincere gratitude to our family members and friends for their constant support and motivation.

Abstract

Sign language is an important visual communication medium used by deaf and hard-of-hearing communities. However, communication can become difficult when the communicating individuals do not share knowledge of the same sign language. Deep learning and computer vision provide an opportunity to develop intelligent systems that can automatically recognize hand signs and convert their meanings into machine-readable representations.

This case study presents a deep learning-based framework for sign language recognition and its extension toward sign-to-text generation. The implemented recognition system uses the American Sign Language (ASL) Alphabet dataset and a transfer-learning-based ResNet50 architecture for multiclass image classification. The input images are resized, augmented, and preprocessed before being provided to the pretrained convolutional neural network. Global average pooling, batch normalization, a fully connected layer, dropout, and a softmax classification layer are used to obtain the final sign prediction.

The system is evaluated using standard classification metrics including accuracy, precision, recall, F1-score, loss, and confusion-matrix analysis. Training and validation curves are also examined to understand convergence and generalization behaviour. The trained classifier can additionally accept a user-provided sign image and predict its corresponding sign class together with the associated confidence score.

To extend isolated sign recognition toward continuous communication, the study incorporates a CNN–LSTM–Attention framework. CNN-based visual features can be extracted from consecutive sign observations and supplied to an LSTM network for temporal modelling. An attention mechanism can assign greater importance to relevant elements of the sequence during the generation process. Thus, the study establishes a structured foundation for developing an assistive system capable of progressing from isolated gesture recognition toward continuous sign-to-text communication.

List of Figures

List of Figures

4.1 Overall Methodology Workflow 17
4.2 CNN–LSTM–Attention Framework for Sequential Sign-to-Text Generation 20

6.1 Training and Validation Accuracy 28
6.2 Training and Validation Loss 28
6.3 Confusion Matrix of Sign Classification 29
6.4 Prediction of a User-Provided Sign Image 30

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY 1979

List of Tables

List of Tables

2.1 Comparison of Deep Learning Models Relevant to the Case Study 11

3.1 Hardware and Computational Requirements 13
3.2 Software and Frameworks Used 14
3.3 ASL Alphabet Dataset Characteristics 14

4.1 Proposed ResNet50 Classification Architecture 19
4.2 Architecture Comparison for the Proposed System 21

5.1 Training Hyperparameters 24

6.1 Final Model Performance 27

8.1 Data Augmentation Strategies 37
8.2 Important Model Hyperparameters 39

10.1 Categories of References Used in the Study 41
10.2 Software Resources Used 43

INSTITUTE OF TECHNOLOGY
1979

Table of Contents

List of Figures		**iv**
List of Tables		**v**
Table of Contents		**vi**
1	**1. Introduction**	**1**
	1.1 Background and Significance	1
	1.2 Deep Learning for Sign Language Recognition	1
	1.3 Problem Definition	2
	1.4 Challenges in Sign Language Recognition	2
	1.4.1 Variation in Hand Appearance	3
	1.4.2 Background and Illumination	3
	1.4.3 Gesture Similarity	3
	1.4.4 Camera Position and Orientation	3
	1.4.5 Static Versus Continuous Sign Language	3
	1.5 Objectives	3
	1.6 Motivation and Report Structure	4
2	**2. Literature Review and Theoretical Study**	**5**
	2.1 Evolution of Sign Language Recognition	5
	2.1.1 Traditional Image-Based Recognition	5
	2.1.2 CNN-Based Recognition	6
	2.1.3 Transfer Learning	6
	2.2 Theoretical Foundations	6
	2.2.1 Artificial Neuron	6
	2.2.2 Multilayer Perceptron	7
	2.2.3 Activation Functions	7
	2.2.4 Classification Loss	8
	2.2.5 Gradient Descent and Optimisation	8
	2.2.6 Regularisation	8

	2.2.7 Convolution and Pooling	8
	2.2.8 Residual Learning	9
	2.3 Sequential Deep Learning	9
	2.3.1 Recurrent Neural Networks	9
	2.3.2 Long Short-Term Memory	9
	2.3.3 Attention Mechanism	10
	2.4 Comparative Analysis of Deep Learning Models	11
	2.5 Synthesis and Open Challenges	11
	2.6 Research Gap	12
3	**3. System Requirements and Specifications**	**13**
	3.1 Hardware and Software Requirements	13
	3.1.1 Hardware Specifications	13
	3.1.2 Software and Frameworks	14
	3.2 Dataset Description and Characteristics	14
	3.3 Data Preprocessing	15
	3.4 Evaluation Metrics	15
	3.4.1 Accuracy	15
	3.4.2 Precision	15
	3.4.3 Recall	16
	3.4.4 F1-Score	16
	3.4.5 Confusion Matrix	16
4	**4. Methodology and System Design**	**17**
	4.1 Methodology Workflow	17
	4.2 Proposed CNN Architecture	18
	4.2.1 ResNet50 Backbone	18
	4.2.2 Transfer Learning	18
	4.2.3 Classification Head	19
	4.3 Sequential Sign-to-Text Architecture	19
	4.3.1 CNN Feature Extraction	20
	4.3.2 LSTM Sequence Modelling	20
	4.3.3 Attention Mechanism	20
	4.3.4 Text Generation	20
	4.4 Model Architecture Comparison	21
5	**5. Implementation Details and Experimental Setup**	**22**
	5.1 Implementation Environment	22

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY 1979

	5.2 Dataset Preparation	22
	5.3 Training Protocol	23
	5.3.1 Initial Training	23
	5.3.2 Fine-Tuning	23
	5.3.3 Loss Function	23
	5.3.4 Optimization and Hyperparameters	24
	5.4 Regularisation and Training Controls	24
	5.4.1 Data Augmentation	24
	5.4.2 Batch Normalization	24
	5.4.3 Dropout	25
	5.4.4 Early Stopping	25
	5.4.5 Model Checkpointing	25
	5.5 Experimental Procedure	25
6	**6. Results and Performance Analysis**	**27**
	6.1 Overview	27
	6.2 Quantitative Performance Metrics	27
	6.2.1 Initial Training Performance	27
	6.2.2 Fine-Tuning Performance	28
	6.3 Training and Validation Analysis	28
	6.4 Confusion Matrix Analysis	29
	6.5 Precision, Recall and F1-Score Analysis	29
	6.6 User-Provided Image Prediction	29
	6.7 Qualitative Analysis	30
	6.8 Sequential Sign-to-Text Analysis	30
	6.9 Robustness and Generalization Analysis	31
	6.10 Summary of Findings	31
7	**7. Conclusion and Future Scope**	**33**
	7.1 Conclusion	33
	7.2 Limitations	33
	7.3 Future Scope	34
	7.3.1 Continuous Video-Based Recognition	34
	7.3.2 Word-Level Sign Recognition	34
	7.3.3 Sentence-Level Translation	34
	7.3.4 Real-Time Webcam Deployment	34
	7.3.5 Mobile and Web Deployment	34
	7.3.6 Multimodal Sign-Language Understanding	35

CHANAKYA BHARATHI INSTITUTE OF TECHNOLOGY 1979

8	**8. Appendix I: Model Details and Preprocessing**	**36**
	8.1 ResNet50 Model Details	36
	8.2 Model Training Strategy	36
	8.3 Preprocessing Pipeline	37
	8.4 Data Augmentation Strategies	37
	8.5 Classification Head Details	38
	8.6 LSTM and Attention Extension	38
	8.7 Important Hyperparameters	39
9	**References**	**40**
	9.1 Reference Categories	41
	9.2 Bibliographic References	41
	9.3 Software and Online Resources	42
	9.4 Reference Summary	42

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY 1979

1. Introduction

1.1 Background and Significance

Sign language is a visual form of communication that conveys information through hand configurations, hand movements, body positions, facial expressions, and other visual cues. It plays an important role in communication for deaf and hard-of-hearing communities. Although sign language provides an effective communication medium among individuals familiar with it, communication barriers can arise when the receiving person does not understand the signs being used.

The increasing availability of image-processing techniques and deep learning architectures has created new possibilities for automated sign language recognition. Instead of manually designing visual features, deep neural networks can learn discriminative representations directly from image data. Convolutional Neural Networks (CNNs), in particular, have demonstrated strong performance in image classification because they can learn hierarchical spatial features.

The present case study investigates a deep learning-based approach for recognizing isolated sign-language images. The system uses transfer learning with ResNet50, a deep residual convolutional network, to extract visual representations from hand-sign images. A classification head is then used to map the extracted features to the corresponding sign classes.

The study also considers the transition from isolated sign recognition to continuous sign-to-text generation. In continuous sign language, individual gestures form temporal sequences. Therefore, recognising a complete expression requires not only spatial understanding of individual frames but also temporal modelling of the sequence. LSTM networks and attention mechanisms provide a suitable framework for this sequential component.

1.2 Deep Learning for Sign Language Recognition

Traditional image-recognition systems commonly rely on manually engineered features such as contours, edges, shape descriptors, skin-colour segmentation, or geometric properties of the hand. Although such methods can work under controlled conditions, their performance

may deteriorate when the lighting, background, orientation, scale, or appearance of the hand changes.

Deep learning reduces the dependence on manually engineered features by learning representations directly from training data. CNNs progressively transform an input image into increasingly abstract feature representations through convolutional layers, non-linear activation functions, and downsampling operations.

For sign-language recognition, these learned representations can capture important characteristics such as finger configuration, hand shape, relative position of fingers, and overall gesture structure. Transfer learning further reduces the computational requirements by allowing a network pretrained on a large image dataset to be adapted to a sign-language classification task.

1.3 Problem Definition

The primary problem addressed in this case study is the automatic recognition of a sign-language gesture from an input image.

Given an RGB image X, the trained classifier produces a probability distribution over K sign classes:

$$P(y = k \mid X), \qquad k = 1, 2, \ldots, K$$

The predicted class is obtained as:

$$\hat{y} = \arg\max_{k} P(y = k \mid X)$$

For the ASL Alphabet classification task considered in this work, the model predicts one of the predefined sign classes.

The broader objective is to extend the isolated classification system toward a sequential framework in which a sequence of visual features is transformed into a meaningful textual representation.

1.4 Challenges in Sign Language Recognition

Several factors make sign-language recognition a challenging computer-vision problem.

1.4.1 Variation in Hand Appearance

Different users may have different hand sizes, proportions, skin tones, and signing styles. A robust model must therefore learn features that are not excessively dependent on a single user's appearance.

1.4.2 Background and Illumination

Images captured in uncontrolled environments may contain complex backgrounds, shadows, reflections, and varying illumination. Such factors can introduce irrelevant visual information.

1.4.3 Gesture Similarity

Certain signs may have similar hand configurations. Small differences in finger position or orientation can determine the correct class, making fine-grained feature extraction important.

1.4.4 Camera Position and Orientation

Changes in camera distance, viewing angle, hand position, and rotation can affect the visual appearance of a gesture.

1.4.5 Static Versus Continuous Sign Language

Static image classification identifies an individual gesture, whereas continuous sign-language understanding requires modelling the temporal relationship between multiple gestures. This difference motivates the inclusion of recurrent sequence models in the proposed framework.

1.5 Objectives

The major objectives of this case study are:

1. To develop a deep learning-based system for isolated sign-language image classification.
2. To preprocess and augment sign-language images so that the model can learn robust visual representations.
3. To apply transfer learning using the pretrained ResNet50 architecture.
4. To construct a classification head capable of predicting multiple sign classes.

5. To evaluate the model using accuracy, loss, precision, recall, F1-score, and confusion-matrix analysis.
6. To test the trained model using user-provided sign images.
7. To study the extension of visual sign recognition toward sequential sign-to-text generation using LSTM and attention.

1.6 Motivation and Report Structure

The motivation behind this work is to investigate how modern deep learning techniques can be combined to address different levels of the sign-language understanding problem.

The report is organised into nine major sections. Chapter 1 introduces the problem and its significance. Chapter 2 discusses the theoretical foundations and relevant research directions. Chapter 3 describes the hardware, software, dataset, and evaluation requirements. Chapter 4 presents the proposed system architecture and methodology. Chapter 5 describes implementation and experimental settings. Chapter 6 analyses the experimental results. Chapter 7 presents conclusions, limitations, and future directions. Chapter 8 provides detailed model and preprocessing information. Chapter 9 contains the references used in the study.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY 1979

2. Literature Review and Theoretical Study

2.1 Evolution of Sign Language Recognition

Sign-language recognition has developed from traditional handcrafted-feature methods toward deep neural architectures. Early approaches often relied on image segmentation, contour extraction, geometric descriptors, or manually selected features. These approaches were useful for controlled environments but required significant task-specific design.

The development of CNNs changed the field by allowing models to learn spatial features directly from images. More recent systems have adopted transfer learning, residual architectures, recurrent networks, attention mechanisms, and multimodal learning.

2.1.1 Traditional Image-Based Recognition

Traditional recognition systems commonly use a sequence of operations such as image acquisition, background removal, hand segmentation, feature extraction, and classification.

Typical handcrafted features include:

- contour and boundary descriptors,
- shape and geometric features,
- edge information,
- colour-based segmentation features,
- histogram-based descriptors.

These methods provide interpretable feature-processing pipelines, but their robustness can be limited when the visual conditions differ from those encountered during development.

2.1.2 CNN-Based Recognition

CNNs learn spatially structured representations through convolutional filters. Early layers generally learn lower-level features such as edges and textures, while deeper layers learn more task-specific representations.

A simplified CNN pipeline is:

Input Image → Convolution → Activation → Pooling → Deep Features → Classifier

The ability to learn these representations automatically makes CNNs suitable for visual sign recognition.

2.1.3 Transfer Learning

Training a deep CNN from random initialization can require large datasets and substantial computational resources. Transfer learning addresses this issue by using representations learned from a large source dataset and adapting them to a target problem.

The transfer-learning process used in this case study can be represented as:

ImageNet Pretrained Model → ResNet50 → Sign-Language Dataset → Fine-Tuning

During the initial stage, the pretrained convolutional feature extractor can be frozen while the newly introduced classification layers are trained. In a subsequent fine-tuning stage, selected deeper layers can be unfrozen and trained using a smaller learning rate.

2.2 Theoretical Foundations

2.2.1 Artificial Neuron

An artificial neuron computes a weighted combination of its inputs followed by an activation function:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

$$a = f(z)$$

where x_i represents an input, w_i represents its weight, b is the bias, and f is an activation function.

2.2.2 Multilayer Perceptron

A Multilayer Perceptron (MLP) consists of an input layer, one or more hidden layers, and an output layer. MLPs can learn nonlinear decision boundaries through the use of nonlinear activation functions.

Although an MLP can classify flattened image features, it does not naturally preserve the two-dimensional spatial structure of an image. CNNs are therefore more suitable for visual recognition.

2.2.3 Activation Functions

Common activation functions include ReLU, sigmoid, and softmax.

The ReLU function is:

$$f(x) = \max(0, x)$$

The sigmoid function is:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

For multiclass classification, the softmax function converts output logits into class probabilities:

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

2.2.4 Classification Loss

For a multiclass classification problem with a single correct class, categorical cross-entropy is commonly used. When integer class labels are provided directly, sparse categorical cross-entropy can be used:

$$L = -\log(P(y_{\text{true}}))$$

The objective of training is to minimise this loss over the training dataset.

2.2.5 Gradient Descent and Optimisation

Neural-network parameters are updated using gradient-based optimisation. The basic gradient-descent update is:

$$\vartheta_{t+1} = \vartheta_t - \eta \nabla_{\vartheta} L(\vartheta_t)$$

where η is the learning rate.

Modern optimisers such as Adam combine adaptive learning-rate behaviour with momentum-like estimates of gradients.

2.2.6 Regularisation

Deep neural networks may overfit their training data. Regularisation methods such as dropout, L1/L2 penalties, data augmentation, and early stopping can reduce this risk.

Dropout randomly deactivates a proportion of activations during training. In this study, dropout is used in the classification head to improve generalisation.

2.2.7 Convolution and Pooling

A convolution operation applies learnable filters to local regions of an image. For an input feature map X and filter W, a simplified convolution operation can be expressed as:

$$Y(i,j) = \sum_m \sum_n W(m,n) X(i+m, j+n)$$

Pooling reduces the spatial dimensions of feature maps and can provide a degree of trans-

lational robustness.

2.2.8 Residual Learning

Very deep CNNs can become difficult to optimise as network depth increases. ResNet introduces residual connections that allow a block to learn a residual mapping:

$$y = F(x, W) + x$$

The shortcut connection enables information and gradients to flow more effectively through deep networks.

2.3 Sequential Deep Learning

2.3.1 Recurrent Neural Networks

RNNs are designed for sequential data. At time step t, a simplified recurrent update is:

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

where x_t is the current input, h_{t-1} is the previous hidden state, and h_t is the current hidden state.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
1979

2.3.2 Long Short-Term Memory

LSTM networks introduce memory cells and gates to improve the learning of long-term dependencies.

The principal components are:

- Forget gate,
- Input gate,
- Cell state,
- Output gate.

This makes LSTM suitable for modelling temporal relationships between consecutive sign representations.

2.3.3 Attention Mechanism

Attention allows a model to assign different importance to different elements of an input sequence.

For a sequence of feature vectors, attention weights can be represented as:

$$\alpha = \frac{\exp(e_t)}{\sum_{j=1}^{T} \exp(e_j)}$$

The context vector can then be computed as:

$$c = \sum_{t=1} \alpha_t h_t$$

This mechanism allows the sequence model to focus more strongly on the visual representations that contribute to the current prediction.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY 1979

2.4 Comparative Analysis of Deep Learning Models

Table 2.1: Comparison of Deep Learning Models Relevant to the Case Study

Model	Primary Role	Strength	Limitation
MLP	Basic classification	Simple and computationally inexpensive	Does not preserve image spatial structure
CNN	Image recognition	Learns hierarchical spatial features	Does not naturally model temporal sequences
VGG16	Deep visual feature extraction	Simple and systematic architecture	Large parameter count
ResNet50	Deep visual feature extraction	Residual connections enable effective deep learning	Higher computational cost than lightweight CNNs
RNN	Sequence modelling	Handles sequential information	Can struggle with long-term dependencies
LSTM	Temporal modelling	Better handling of long-term dependencies	More computationally expensive
BiLSTM	Bidirectional sequence modelling	Uses information from both temporal directions	Requires the complete sequence
CNN–LSTM	Spatial + temporal modelling	Combines visual and sequential learning	More complex training pipeline
CNN–LSTM–Attention	Spatial + temporal + selective focus	Attention can emphasise important sequence elements	Requires suitable sequential training data

2.5 Synthesis and Open Challenges

The literature and theoretical study indicate that isolated sign recognition and continuous sign-language understanding are related but distinct tasks. CNN-based models are effective for extracting spatial information from individual images, whereas recurrent architectures are more appropriate when temporal relationships between consecutive observations must be modelled.

Important open challenges include robustness to real-world conditions, continuous gesture segmentation, signer independence, large-scale vocabulary recognition, temporal alignment, and the integration of non-manual components such as facial expressions and body posture.

2.6 Research Gap

A static image classifier can identify an isolated gesture but does not by itself provide complete sentence-level sign-language translation. Continuous communication requires the combination of spatial feature extraction and temporal sequence modelling.

The present case study therefore establishes a two-level framework: an implemented ResNet50-based static sign-recognition component and an LSTM-attention-based sequential extension for future continuous sign-to-text generation.

CHAITANYA BHARATHI
INSTITUTE OF TECHNOLOGY
1979

3. System Requirements and Specifications

3.1 Hardware and Software Requirements

The development and experimentation for this case study are performed using a cloud-based deep learning environment. Google Colab provides access to Python-based development tools and, when available, GPU acceleration for neural-network training.

3.1.1 Hardware Specifications

Table 3.1: Hardware and Computational Requirements

Component	**Requirement / Description**
Development Platform	Google Colaboratory
Processor	Cloud-provided CPU
GPU	NVIDIA GPU when available in the Colab runtime
Memory	Runtime-dependent system memory
Storage	Temporary Colab storage / connected cloud storage
Input Device	Image or webcam source for future deployment
Input Type	RGB sign-language image
Model Input Size	$224 \times 224 \times 3$

3.1.2 Software and Frameworks

Table 3.2: Software and Frameworks Used

Software / Library	Purpose
Python	Core programming language
Google Colab	Development and model-training environment
TensorFlow	Deep learning framework
Keras	Neural-network modelling API
NumPy	Numerical computation
Pandas	Data handling and analysis
Matplotlib	Training-curve visualization
Seaborn	Statistical visualization
Scikit-learn	Evaluation metrics and analysis
Kaggle API	Dataset acquisition
OpenCV	Potential future real-time image/video processing

3.2 Dataset Description and Characteristics

The ASL Alphabet dataset is used for the static sign-recognition component. The dataset contains images representing the English alphabet in American Sign Language together with additional classes corresponding to *space*, *delete*, and *nothing*.

The dataset provides a practical multiclass image-classification problem in which each image is associated with a predefined sign label.

Table 3.3: ASL Alphabet Dataset Characteristics

Property	Description
Dataset	ASL Alphabet
Task	Multiclass image classification
Image Type	RGB
Original Image Format	Colour images
Model Input	$224 \times 224 \times 3$
Number of Classes	29
Alphabet Classes	A–Z
Additional Classes	Space, Delete, Nothing
Learning Objective	Sign-class prediction

3.3 Data Preprocessing

The preprocessing pipeline converts the original images into a format suitable for the ResNet50 network.

The major stages are:

1. Load the image and its corresponding class label.
2. Convert the image into RGB representation where required.
3. Resize the image to 224×224 pixels.
4. Apply training-time data augmentation.
5. Apply the preprocessing expected by the ResNet50 model.
6. Form batches for efficient training.
7. Supply the processed images to the neural network.

Data augmentation can include controlled transformations such as rotation, zoom, translation, and contrast variation. These transformations expose the model to different visual appearances while preserving the underlying sign class.

3.4 Evaluation Metrics

The system is evaluated using several classification metrics.

3.4.1 Accuracy

Accuracy represents the proportion of correctly classified samples:

$$Accuracy = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

3.4.2 Precision

Precision measures the proportion of predicted positive instances that are actually positive:

$$Precision = \frac{TP}{TP + FP}$$

3.4.3 Recall

Recall measures the proportion of actual positive instances that are correctly identified:

$$Recall = \frac{TP}{TP + FN}$$

3.4.4 F1-Score

The F1-score combines precision and recall:

$$F1 = 2\frac{Precision \times Recall}{Precision + Recall}$$

3.4.5 Confusion Matrix

For multiclass classification, a confusion matrix provides a class-by-class comparison between the actual and predicted labels. The diagonal represents correct classifications, while off-diagonal entries indicate misclassifications.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY 1979

4. Methodology and System Design

4.1 Methodology Workflow

The complete methodology is organised into the following stages:

1. Dataset acquisition.
2. Dataset inspection and class verification.
3. Image preprocessing.
4. Data augmentation.
5. Transfer learning using pretrained ResNet50.
6. Construction of the classification head.
7. Initial model training.
8. Fine-tuning of selected ResNet50 layers.
9. Evaluation using quantitative metrics.
10. Prediction on user-provided images.
11. Extension toward sequential CNN–LSTM–Attention modelling.

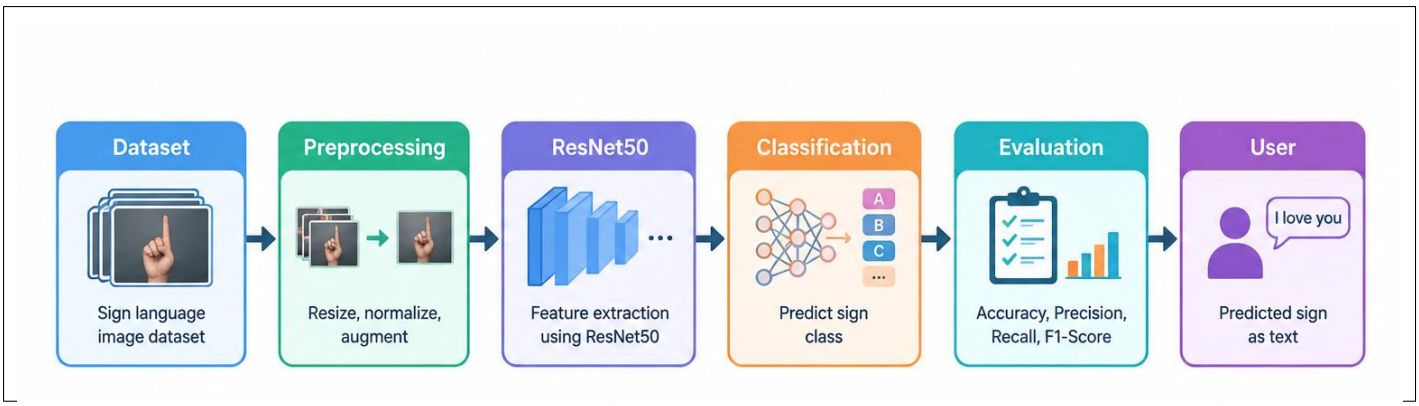


Figure 4.1: Overall Methodology Workflow

4.2 Proposed CNN Architecture

The visual recognition component is based on ResNet50 with ImageNet-pretrained weights. The pretrained network serves as the feature extractor, while a task-specific classification head is attached to predict the sign classes.

The overall classification architecture is:

Input Image → ResNet50 → Global Average Pooling → Batch Normalization → Dense → Dropout → Soft

4.2.1 ResNet50 Backbone

ResNet50 is a deep residual convolutional neural network composed of residual blocks. The residual formulation allows the network to learn a residual mapping while maintaining a short-cut connection.

The backbone is initialized using pretrained ImageNet weights. The early training phase can keep the backbone frozen so that the newly added classification layers learn the target task without modifying the pretrained feature representations.

4.2.2 Transfer Learning

Transfer learning is used to exploit previously learned visual representations.

The procedure consists of:

1. Load ResNet50 with pretrained ImageNet weights.
2. Remove the original ImageNet classification head.
3. Add a task-specific classification head.
4. Freeze the pretrained backbone during initial training.
5. Train the new classification layers.
6. Unfreeze a selected number of deeper layers.
7. Fine-tune the model using a smaller learning rate.

CHULALONGKORN INSTITUTE OF TECHNOLOGY 1979

4.2.3 Classification Head

The classification head consists of:

Global Average Pooling → Batch Normalization → Dense(256, ReLU) → Dropout(0.5) → Dense(29, Softm

Global average pooling converts the final spatial feature maps into a compact feature vector. Batch normalization stabilizes feature distributions. The dense layer learns task-specific representations, while dropout provides regularization. Finally, the softmax layer produces the class probabilities.

Table 4.1: Proposed ResNet50 Classification Architecture

Stage	**Layer**	**Purpose**
Input	$224 \times 224 \times 3$	Input RGB sign image
Feature Extractor	ResNet50	Deep visual feature extraction
Feature Reduction	Global Average Pooling	Spatial-to-vector conversion
Normalization	Batch Normalization	Stabilization and normalization
Fully Connected	Dense(256, ReLU)	Task-specific representation
Regularization	Dropout(0.5)	Reduction of overfitting
Output	Dense(29, Softmax)	Sign-class probability distribution

4.3 Sequential Sign-to-Text Architecture

Static sign recognition identifies one image at a time. To move toward sentence generation, multiple visual observations must be treated as a sequence.

The proposed sequential extension is:

Sign Frames → CNN Features → LSTM → Attention → Text Decoder

Architecture Placeholder

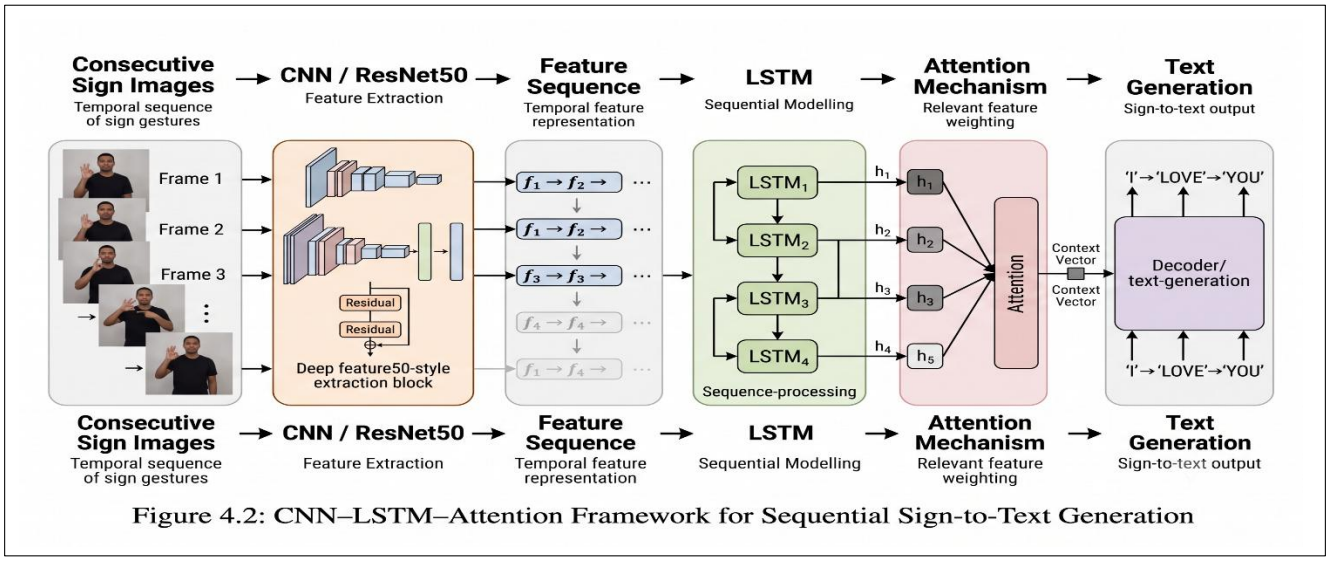


Figure 4.2: CNN–LSTM–Attention Framework for Sequential Sign-to-Text Generation

Figure 4.2: CNN–LSTM–Attention Framework for Sequential Sign-to-Text Generation

4.3.1 CNN Feature Extraction

Each sign frame can be passed through the CNN feature extractor to obtain a compact visual representation. For a sequence containing T observations, the result is:

$$X = [x_1, x_2, \ldots, x_T]$$

where each x_t represents the visual feature vector extracted from the t-th observation.

4.3.2 LSTM Sequence Modelling

The feature sequence is supplied to the LSTM network. The LSTM maintains hidden and cell states that allow information from previous observations to influence later predictions.

This is particularly useful when the meaning of a sequence depends on the order in which gestures occur.

4.3.3 Attention Mechanism

The attention layer assigns weights to the hidden states generated by the LSTM. This enables the model to focus on the most relevant parts of the sign sequence during text generation.

4.3.4 Text Generation

The output of the attention mechanism can be supplied to a decoder that generates textual tokens sequentially.

A conceptual generation process is:

Sign Sequence → Visual Features → Temporal Representation → Attention Context → Text Tokens

The current implementation primarily evaluates the static image-recognition component. Full sentence-level generation requires sequential sign/video data and corresponding sentence annotations.

4.4 Model Architecture Comparison

Table 4.2: Architecture Comparison for the Proposed System

Architecture	**Spatial Learning**	**Temporal Learning**	**Application**
MLP	Limited	No	Basic classification
CNN	Yes	No	Static sign recognition
VGG16	Yes	No	Transfer-learning baseline
ResNet50	Yes	No	Proposed visual recognition
CNN–LSTM	Yes	Yes	Continuous sign recognition
CNN–LSTM–Attention	Yes	Yes	Sequential sign-to-text framework

5. Implementation Details and Experimental Setup

5.1 Implementation Environment

The system is implemented in Python using Google Colaboratory. TensorFlow and Keras are used for model construction and training. NumPy and Pandas support numerical and data-processing operations, while Matplotlib and Seaborn are used for visualisation. Scikit-learn is used for classification metrics and confusion matrix analysis.

The dataset is obtained through the Kaggle dataset interface and prepared for TensorFlow-based image loading and training.

5.2 Dataset Preparation

The dataset is organised into class-specific image directories. Images are loaded with their associated class labels and converted into TensorFlow datasets.

The data pipeline performs:

1. Image loading.
2. Label assignment.
3. Resizing.
4. Training/validation separation.
5. Data augmentation for training samples.
6. ResNet50 preprocessing.
7. Batch formation.
8. Prefetching for improved input-pipeline efficiency.

5.3 Training Protocol

The training procedure consists of two main phases.

5.3.1 Initial Training

During the initial stage, the pretrained ResNet50 feature extractor is frozen. The newly added classification head is trained for the sign-recognition task.

The loss function used is sparse categorical cross-entropy:

$$L = -\log(P(y_{\text{true}}))$$

The Adam optimiser is used for gradient-based parameter updates.

5.3.2 Fine-Tuning

After initial training, selected deeper layers of ResNet50 can be made trainable. Fine-tuning is performed using a smaller learning rate so that the pretrained representations are adapted gradually to the target dataset.

5.3.3 Loss Function

Sparse categorical cross-entropy is selected because each image has one integer-encoded class label.

For an observation whose correct class is y, the loss is:

$$L = -\log(p_y)$$

where p_y is the predicted probability assigned to the correct class.

5.3.4 Optimization and Hyperparameters

Table 5.1: Training Hyperparameters

Parameter	Value / Configuration
Backbone	ResNet50
Pretrained Weights	ImageNet
Input Size	$224 \times 224 \times 3$
Output Classes	29
Initial Optimizer	Adam
Initial Learning Rate	10^{-3}
Fine-Tuning Optimizer	Adam
Fine-Tuning Learning Rate	10^{-5}
Classification Loss	Sparse Categorical Cross-Entropy
Dense Layer	256 units, ReLU
Dropout	0.5
Output Activation	Softmax

Important: The final epoch count and batch size should be reported exactly as used in the final Colab execution. Do not replace these values with assumed values.

5.4 Regularisation and Training Controls

Several mechanisms are incorporated to improve training stability and generalisation.

5.4.1 Data Augmentation

Training images can be subjected to controlled transformations including rotation, zoom, translation, and contrast adjustment. These transformations increase visual diversity.

5.4.2 Batch Normalization

Batch normalization is used in the classification head to normalize intermediate activations and support stable training.

5.4.3 Dropout

A dropout rate of 0.5 is applied before the final classification layer. This prevents the classifier from relying excessively on a small subset of learned activations.

5.4.4 Early Stopping

Early stopping can terminate training when validation performance does not improve for a specified number of epochs. The best available weights can then be restored.

5.4.5 Model Checkpointing

A model checkpoint is maintained so that the model corresponding to the best validation performance can be preserved independently of the final training epoch.

5.5 Experimental Procedure

The experimental procedure follows these stages:

1. Load and inspect the dataset.
2. Verify the number of classes and class labels.
3. Create training and validation datasets.
4. Apply preprocessing and augmentation.
5. Load ImageNet-pretrained ResNet50.
6. Attach the classification head.
7. Train the classification head.
8. Evaluate validation performance.
9. Fine-tune selected backbone layers.
10. Re-evaluate the model.
11. Generate accuracy and loss curves.
12. Generate a confusion matrix.
13. Compute precision, recall, and F1-score.

14. Test a user-provided sign image.

6. Results and Performance Analysis

6.1 Overview

The trained model is evaluated using the validation data and additional inference tests. The evaluation focuses on both overall classification performance and class-level behaviour.

The principal analyses include training and validation curves, classification metrics, confusion-matrix analysis, and prediction of a user-provided sign image.

6.2 Quantitative Performance Metrics

The final numerical values in this section must be taken directly from the final Colab execution. They should not be estimated or copied from unrelated experiments.

Table 6.1: Final Model Performance

Metric	**Final Value**
Training Accuracy	**[0.4429]**
Validation Accuracy	**[97.51 %]**
Training Loss	**[0.8595]**
Validation Loss	**[0.0725]**
Precision	**[0.9751]**
Recall	**[0.9745]**
F1-Score	**[0.9746]**

6.2.1 Initial Training Performance

The initial training phase evaluates the ability of the newly introduced classification head to adapt the pretrained ResNet50 representations to the sign-language dataset.

The training and validation curves should be examined together. Increasing training accuracy accompanied by increasing validation accuracy indicates effective learning, whereas a

widening gap between the two curves can indicate overfitting.

6.2.2 Fine-Tuning Performance

During fine-tuning, selected deeper layers of ResNet50 are unfrozen and trained using a smaller learning rate. This allows the visual feature extractor to adapt more specifically to the sign-language domain.

The effect of fine-tuning should be determined by comparing the validation metrics before and after fine-tuning.

6.3 Training and Validation Analysis

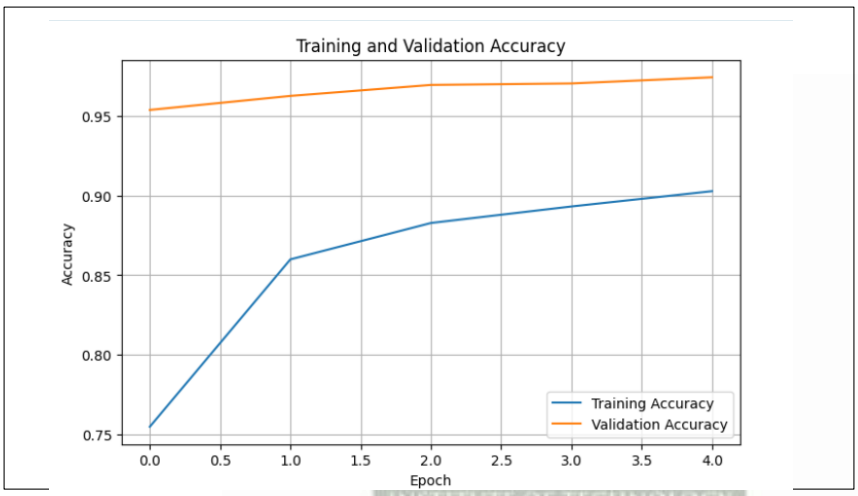


Figure 6.1: Training and Validation Accuracy

The accuracy graph should be used to analyse the convergence behaviour of the model. Training accuracy measures performance on the samples used for optimisation, whereas validation accuracy provides an indication of generalisation to unseen samples.

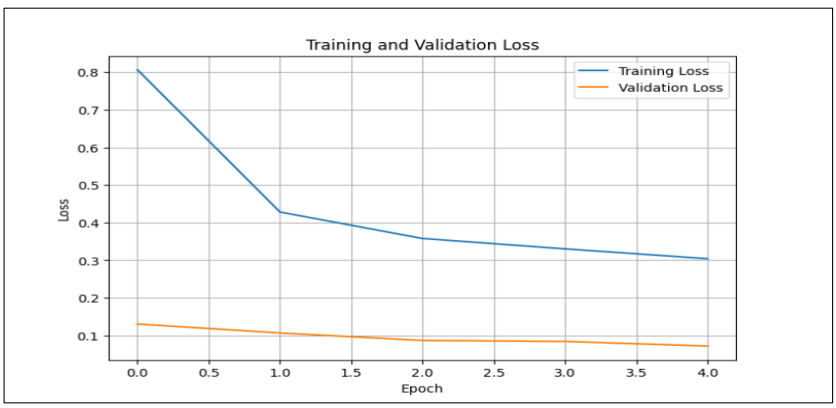


Figure 6.2: Training and Validation Loss

The loss curves indicate how the optimisation objective changes during training. A decreasing validation loss generally indicates improving generalisation, while an increasing validation loss combined with decreasing training loss can indicate overfitting.

6.4 Confusion Matrix Analysis

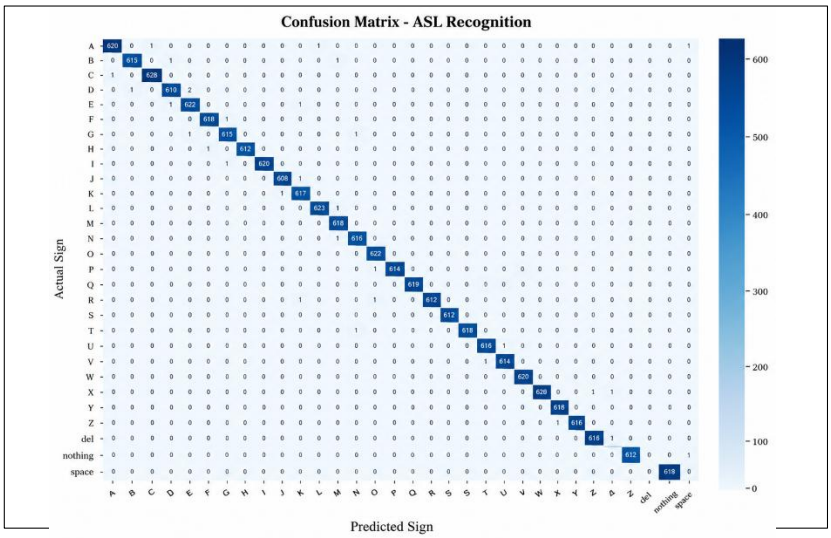


Figure 6.3: Confusion Matrix of Sign Classification

The confusion matrix provides a detailed class-level view of classification behaviour. Strong diagonal values correspond to correct classifications. Off-diagonal values identify sign classes for which the visual representations are more difficult to distinguish.

6.5 Precision, Recall and F1-Score Analysis

Precision, recall, and F1-score provide additional information beyond overall accuracy.

Precision is particularly useful for determining whether a predicted class is reliable. Recall indicates the ability of the model to identify samples belonging to a particular class. The F1-score provides a combined measure of precision and recall.

For a multiclass problem, macro-averaged metrics can be used to give equal importance to each class.

6.6 User-Provided Image Prediction

One of the practical objectives of the system is to classify a sign image supplied by the user.

The inference pipeline is:

User Image → Resize → Preprocess → ResNet50 → Softmax → Predicted Sign

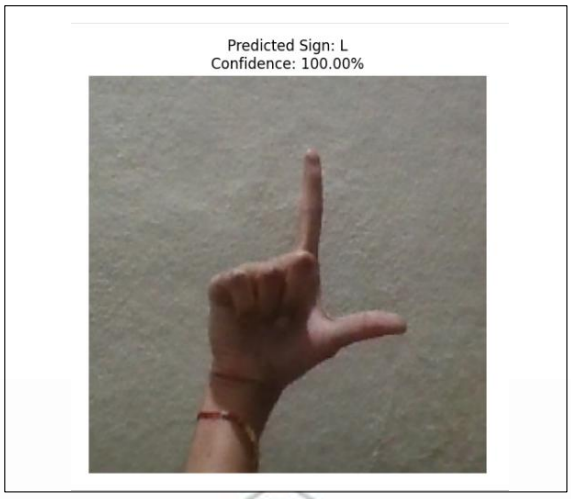


Figure 6.4: Prediction of a User-Provided Sign Image

For each input image, the softmax layer produces probabilities for all classes. The class with the maximum probability is selected as the predicted sign.

6.7 Qualitative Analysis

The qualitative behaviour of the model can be examined using correctly and incorrectly classified examples.

Images with clear hand configurations and visual characteristics similar to the training distribution are expected to be easier to classify. Errors may occur when signs have similar visual configurations or when images contain significant differences in orientation, illumination, scale, or background.

Such observations are useful for identifying areas where additional data augmentation, improved preprocessing, or more diverse training data may be required.

6.8 Sequential Sign-to-Text Analysis

The CNN-based classifier provides the visual recognition component of the proposed framework. For continuous sign-language translation, the output of the visual encoder must be rep-

resented as a sequence.

Conceptually:

$$x_1, x_2, \ldots, x_T \rightarrow f_1, f_2, \ldots, f_T \rightarrow \text{LSTM} \rightarrow \text{Attention} \rightarrow \text{Text}$$

where x_t represents an input observation and f_t represents its visual feature representation.

The LSTM models dependencies between consecutive observations, while attention allows the system to assign different importance to different temporal positions.

The current static-image experiment should not be interpreted as a complete sentence-level translation system. Full sentence generation requires sequential sign/video data with corresponding text annotations and a separately trained sequence-generation component.

6.9 Robustness and Generalization Analysis

Generalization should be evaluated by comparing training and validation behaviour rather than relying only on training accuracy.

Potential sources of distribution shift include:

- different users,
- different camera devices,
- lighting changes,
- different backgrounds,
- hand orientation,
- image quality,
- distance between hand and camera.

A model that performs well only on images closely matching the training distribution may not necessarily provide the same performance in real-world conditions.

6.10 Summary of Findings

The experimental analysis should summarise:

1. Overall classification accuracy.
2. Validation loss and convergence behaviour.
3. Precision, recall, and F1-score.
4. Classes with comparatively stronger performance.
5. Classes that exhibit comparatively higher confusion.
6. Effect of fine-tuning on validation performance.
7. Behaviour on user-provided images.
8. Practical limitations of static image recognition.



7. Conclusion and Future Scope

7.1 Conclusion

This case study investigated a deep learning-based approach to sign-language recognition using transfer learning and a ResNet50 visual feature extractor. The system processes RGB sign images, applies preprocessing and augmentation, extracts deep visual features, and predicts one of the predefined sign classes through a task-specific classification head.

The use of transfer learning provides a practical method for adapting a deep convolutional architecture to sign-language recognition without training the complete network from random initialisation. Global average pooling, batch normalization, a fully connected layer, dropout, and softmax are used to construct the classification head.

The system is evaluated using accuracy, loss, precision, recall, F1-score, confusion-matrix analysis, and training-validation curves. User-provided image inference provides an additional demonstration of the practical use of the trained classifier.

The study also establishes the architectural foundation required to progress from isolated sign recognition to continuous sign-to-text generation. CNN-based feature extraction can be combined with LSTM sequence modelling and attention to process temporal sign sequences. However, complete sentence-level translation requires appropriately annotated sequential or video datasets and dedicated sequence-to-text training.

7.2 Limitations

The implemented recognition component primarily addresses isolated static sign classification. Therefore, it does not by itself provide complete continuous sign-language translation.

The ASL Alphabet dataset also represents a controlled dataset setting. Real-world performance may differ because of changes in background, lighting, camera quality, hand orientation, user appearance, and signing style.

Another limitation is that the current image-based system focuses mainly on hand-gesture

information. Natural sign language can also contain facial expressions, body posture, head movement, and other non-manual signals.

Finally, sentence-level sign-to-text generation requires sequential training examples paired with natural-language text. Such data are different from isolated alphabet images and require a dedicated temporal learning pipeline.

7.3 Future Scope

The proposed framework can be extended in several directions.

7.3.1 Continuous Video-Based Recognition

Instead of processing isolated images, a webcam or video stream can be divided into frames. Each frame can be processed by the CNN feature extractor and the resulting sequence can be supplied to an LSTM.

Video → Frames → CNN → LSTM → Attention → Text

7.3.2 Word-Level Sign Recognition

The vocabulary can be expanded beyond alphabetic signs to include complete words and common expressions.

7.3.3 Sentence-Level Translation

A sequence-to-sequence architecture can be trained using sign-language video sequences paired with corresponding textual sentences.

7.3.4 Real-Time Webcam Deployment

The trained model can be integrated with OpenCV to provide real-time sign recognition from a webcam.

7.3.5 Mobile and Web Deployment

The trained model can be converted into a deployment-friendly format such as TensorFlow Lite or integrated into a web application for wider accessibility.

7.3.6 Multimodal Sign-Language Understanding

Future systems can combine hand gestures with facial expressions, body pose, and other non-manual components to obtain a more complete representation of natural sign language.

CHAITANYA BHARATHI
INSTITUTE OF TECHNOLOGY
1979

8. Appendix I: Model Details and Preprocessing

8.1 ResNet50 Model Details

The implemented visual recognition architecture uses ResNet50 with ImageNet-pretrained weights.

The high-level structure is:

Input : $224 \times 224 \times 3$

↓

ResNet50 Backbone

↓

Global Average Pooling

↓

Batch Normalization

↓

Dense(256, ReLU)

↓

Dropout(0.5)

↓

Dense(29, Softmax)

8.2 Model Training Strategy

The model uses a two-stage transfer-learning strategy.

1. Load ResNet50 with ImageNet pretrained weights.
2. Freeze the pretrained feature extractor.

3. Train the newly added classification head.
4. Evaluate validation performance.
5. Unfreeze selected deeper layers.
6. Reduce the learning rate.
7. Fine-tune the selected layers.
8. Save the model corresponding to the best validation performance.

8.3 Preprocessing Pipeline

The preprocessing pipeline is:

Raw Image → RGB Conversion → Resize → Augmentation → ResNet50 Preprocessing → Model Input

The model expects images in the spatial resolution required by ResNet50. Consistent preprocessing between training and inference is essential because the network learns representations from the distribution produced by the preprocessing pipeline.

8.4 Data Augmentation Strategies

The training pipeline can use controlled image transformations to increase visual diversity.

Table 8.1: Data Augmentation Strategies

Augmentation	Purpose
Rotation	Simulates small orientation variations
Zoom	Simulates changes in hand-to-camera distance
Translation	Simulates changes in hand position
Contrast Variation	Improves robustness to illumination differences

Augmentation should be applied only to the training data. Validation data should represent the natural evaluation distribution and should not receive random training transformations.

8.5 Classification Head Details

The classification head converts the high-dimensional visual representation produced by ResNet50 into class probabilities.

$$z = Wh + b$$

where h is the learned feature representation. The final softmax layer converts the output logits into probabilities:

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^{29} e^{z_j}}$$

The predicted sign is:

$$\hat{y} = \arg\max_i P_i$$

8.6 LSTM and Attention Extension

For continuous sign recognition, each frame can be represented by a feature vector:

$$F = [f_1, f_2, \ldots, f_T]$$

The LSTM processes this sequence and produces hidden states:

$$H = [h_1, h_2, \ldots, h_T]$$

An attention mechanism computes weights α_t and produces a weighted context vector:

$$c = \sum_{t=1} \alpha_t h_t$$

The context vector can then be used by a decoder to generate text tokens.

This appendix describes the proposed sequential extension; a complete trained implementation requires an appropriately annotated sequential sign-language dataset.

8.7 Important Hyperparameters

Table 8.2: Important Model Hyperparameters

Parameter	Configuration
Backbone	ResNet50
Pretrained Dataset	ImageNet
Input Resolution	224×224
Number of Classes	29
Hidden Dense Units	256
Dense Activation	ReLU
Dropout Rate	0.5
Output Activation	Softmax
Initial Optimizer	Adam
Initial Learning Rate	10^{-3}
Fine-Tuning Learning Rate	10^{-5}
Loss Function	Sparse Categorical Cross-Entropy

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY 1979

9. References

The references used for this report cover the theoretical foundations of deep learning, convolutional neural networks, transfer learning, residual networks, recurrent neural networks, long short-term memory networks, and attention mechanisms. These sources were used to understand the concepts and techniques applied in the proposed sign language recognition system.

9.1 Reference Categories

The major categories of references consulted for this work are summarized below.

Table 10.1: Categories of References Used in the Study

Category	**Purpose**
Deep Learning	Fundamental concepts including neural networks, optimisation, activation functions, loss functions and regularisation.
Convolutional Neural Networks	Understanding convolution, pooling, feature extraction and image classification.
Transfer Learning	Understanding the use of pretrained models for image recognition tasks and fine-tuning strategies.
ResNet50	Study of residual learning and the architecture of deep residual networks used as the CNN backbone.
Recurrent Neural Networks	Understanding sequential modelling for handling sequences of recognised sign features.
LSTM Networks	Understanding memory-based sequence modelling and long-term dependency handling.
Attention Mechanism	Understanding selective weighting of relevant features during sequence processing and generation.
Sign Language Recognition	Study of computer vision and deep learning approaches applied to automatic sign language recognition.

9.2 Bibliographic References

1. LeCun, Y., Bengio, Y., and Hinton, G., "Deep Learning," *Nature*, vol. 521, pp. 436–444, 2015.

2. Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*, MIT Press, 2016.

3. He, K., Zhang, X., Ren, S., and Sun, J., "Deep Residual Learning for Image Recognition," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

4. Simonyan, K. and Zisserman, A., "Very Deep Convolutional Networks for Large-Scale Image Recognition," arXiv preprint arXiv:1409.1556, 2014.

5. Hochreiter, S. and Schmidhuber, J., "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

6. Bahdanau, D., Cho, K., and Bengio, Y., "Neural Machine Translation by Jointly Learning to Align and Translate," arXiv preprint arXiv:1409.0473, 2014.

7. Vaswani, A. et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

8. Shorten, C. and Khoshgoftaar, T. M., "A Survey on Image Data Augmentation for Deep Learning," *Journal of Big Data*, vol. 6, no. 60, 2019.

9. Chollet, F., *Deep Learning with Python*, Manning Publications, 2021.

10. Bishop, C. M., *Pattern Recognition and Machine Learning*, Springer, 2006.

9.3 Software and Online Resources

The implementation and experimental study were carried out using Python-based deep learning tools and cloud computing resources. The major software resources used during the implementation include Python, TensorFlow/Keras, NumPy, Pandas, Matplotlib and Scikit-learn.

Table 10.2: Software Resources Used

Resource	**Application in the Project**
Python	Model development and experimentation
TensorFlow / Keras	Deep learning model implementation and training
NumPy	Numerical and array-based operations
Pandas	Dataset organisation and preprocessing
Matplotlib	Training and evaluation visualisation
Scikit-learn	Evaluation metrics and performance analysis
Google Colab	Cloud-based development and model experimentation

9.4 Reference Summary

The references collectively provide the theoretical and practical foundation for the proposed system. Deep learning literature supports the CNN-based image recognition component, while research on residual networks provides the basis for using ResNet50 as a pretrained feature extractor. Literature on LSTM networks and attention mechanisms provides the foundation for extending individual sign recognition toward sequential sign-to-text generation.

These references therefore support the major stages of the proposed workflow, including image preprocessing, feature extraction, transfer learning, classification, sequential modelling and attention-based sequence generation.